CGE-P LAB NOTES

Lab 0.1: Prerequisites and Credential Setup

Do this once, before Lab 2.2. It takes about 90 minutes and it is the difference between labs that work and an afternoon of authentication errors.

Three things happen here: you build a sandbox where mistakes are cheap, you set up credentials that are short-lived by default, and you put a spending cap on it before you deploy anything that bills.

For reading. Code lines wrap to fit the page; copy code from docs/00_01_prerequisites.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/00_01_prerequisites.md` in your clone, not from the PDF.

Read this first: these labs cost money

This curriculum uses customer-managed KMS keys, which is where most of the cost comes from and is the point. Rough numbers for a single learner:

If you	Expect
Destroy each lab the same day	Under **$5** for the whole curriculum
Leave the state backend up for a month (recommended)	**$1/month** for its CMK
Leave Security Hub + Config + a few keys running a month	**$10 to $15**
Forget to destroy Lab 5.2 for a quarter	**$30 to $45**

Two specific traps:

- **KMS keys bill $1/month each, whether used or not**, and they keep billing through the 7-to-30 day deletion window after you schedule deletion. You cannot delete one immediately. Each lab tells you which keys it creates.
- **CloudTrail data events** bill per event. Lab 5.2 scopes them to one bucket for this reason. Point them at a busy bucket and the number changes character.

Set the budget alarm in Step 5 before you apply anything. It is not optional, and it is the only step here that has ever saved anyone real money.

Learning objectives

- Stand up an AWS sandbox account isolated from anything you care about.
- Authenticate with short-lived credentials instead of long-lived access keys, and explain why that distinction is a control.
- Stand up a GCP project with the right APIs and both authentication modes.
- Install and verify the full toolchain.
- Cap your own spending before you can hurt yourself.

Part 1: The AWS sandbox

Step 1 Use a separate account

Not your production account. Not the account with your domain in it. A separate AWS account, because every lab ends with `terraform destroy` and the blast radius of a mistake should be a sandbox.

Two ways to get one:

Approach	Do this if
New member account in an Organization (recommended)	You have or can create an AWS Organization. Gives you a clean account with consolidated billing and the option to attach an SCP later.
A brand new standalone account	You have no Organization and do not want one. Works fine. You manage its billing separately.

If you have no AWS account at all

Sign up at **https://aws.amazon.com** and choose **Create an AWS Account**. You will need three things:

- **An email address** nobody has used for an AWS account before. If you plan to make more than one account later, use the `you+cgep-lab@example.com` form now, because AWS requires a unique address per account and most mail providers deliver plus-addressed mail to the same inbox.
- **A payment card.** AWS asks for one even on the free tier and places a small temporary authorization on it, usually about a dollar, which it releases.
- **A phone** for verification.

Choose the **Basic support plan**, which is free. Everything in this curriculum works on it.

The email and password you sign up with become the **root user** of the new account. That account can do anything, including closing itself and changing billing, so Step 2 immediately locks it down and you never use it again for day to day work.

About the free tier. New accounts get twelve months of free-tier allowances plus some always-free services. It does not cover everything this curriculum uses. KMS keys are about a dollar per month each and are not free-tier eligible, so Step 5's budget alarm is doing real work, not ceremony.

If you already have an AWS Organization

Creating a member account in an existing Organization:

```
aws organizations create-account \
  --email you+cgep-lab@example.com \
  --account-name "CGE-P Lab Sandbox" \
  --profile YOUR_MGMT_PROFILE
```

```
# Poll until State is SUCCEEDED, then note the AccountId
aws organizations list-create-account-status \
  --states SUCCEEDED --profile YOUR_MGMT_PROFILE
```

The `you+cgep-lab@example.com` form matters: AWS requires a unique email per account, and most mail providers deliver plus-addressed mail to the same inbox.

> **A caution about Organizations.** A member account created this way has an `OrganizationAccountAccessRole` your management account can assume, and no root password until you reset it. That is convenient and it is also why Lab 5.2 may find Config blocked by a service control policy. Both are normal.

Step 2 Harden the root user

Do this before anything else, in the console, signed in as root:

1. **Enable MFA on the root user.** A hardware key or an authenticator app.
2. **Delete any root access keys.** There should be none. If there are, delete them; nothing should ever use root programmatically.
3. **Set a strong unique password** in your password manager.
4. Then sign out of root and do not use it again. Everything below uses a role.

That is IA-2(1), AC-6(5), and IA-5 in three minutes, and it is the first thing any assessor checks.

Step 3 Create a non-root IAM user

Do this before you touch credentials, because the credential step will otherwise hand you root and not warn you.

In the console: **IAM → Users → Create user**.

- Name it `cgep-lab`.
- Tick **"Provide user access to the AWS Management Console"**.
- Set a password. Untick the force-reset box if you would rather not.
- Permissions: **Attach policies directly** → `AdministratorAccess`. This is a sandbox; scoping it properly is a week of policy authoring and teaches you nothing this curriculum is about.

Then **Security credentials → Assign MFA device** on that user. If you put a hardware key on root in Step 2, register the same one here. A single key enrols against multiple identities, and AWS allows eight devices per user.

Do not create an access key. A console password is all you need. Step 4 turns the resulting console session into short-lived credentials.

Now sign out of root and sign back in as `cgep-lab` at `https://<ACCOUNT_ID>.signin.aws.amazon.com/console`.

Step 4 Credentials: three ways, ranked

	What it is	Secret on disk	Verdict
`aws login`	Converts your existing console session into temporary credentials that auto-refresh	None	**Use this**
IAM Identity Center	Federated sign-in with short-lived session credentials	None	Right for org-managed accounts, more setup
IAM user access key	A permanent `AKIA…` key and secret	Yes, forever	Last resort

The third option is the credential type Labs 4.3 and 5.4 exist to eliminate. Creating one to work through a curriculum about credential hygiene is a choice you will have to defend to yourself.

The recommended path

```
aws login
```

It prompts for a region (`us-east-1`), opens a browser, and asks which console session to use.

> **Read the session it offers.** `aws login` reuses whatever console session you are signed into, and it will happily offer you **root** without flagging that as dangerous. Root cannot be constrained by any IAM policy and cannot be scoped down afterward. If the browser shows `root`, click **"Sign into new session"** and sign in as the `cgep-lab` user from Step 3.

Then confirm which identity you actually got:

```
aws sts get-caller-identity
```

You want an `Arn` ending in `:user/cgep-lab`. **If it ends in `:root`, stop and redo the sign-in before applying anything.** Half of what this curriculum builds is meaningless under root: Lab 4.3's OIDC role is scoped so CI can plan but not apply, Lab 2.5's vault denies `s3:DeleteBucket` to everyone except the account root, and Lab 2.2 grants the account root key administration as a safety net. Run everything as root and all three become decorative.

`aws login` writes no `~/.aws/credentials` file. It stores a session reference and refreshes automatically while the refresh token is valid. When it lapses, run `aws login` again.

The step everyone misses

Terraform cannot read what `aws login` writes. Nor what Identity Center writes. Both store a session reference; the provider's Go SDK wants materialized credentials. It will tell you this in the least helpful way available:

```
Error: No valid credential sources found
Error: failed to refresh cached credentials, no EC2 IMDS role found,
       operation error ec2imds: GetMetadata, request canceled,
       context deadline exceeded
```

Nothing is wrong with EC2. That is simply where the credential chain gave up. The bridge is a profile that resolves credentials by shelling out to the AWS CLI. Add this to `~/.aws/config`:

```
[profile cgep]
region = us-east-1
credential_process = aws configure export-credentials --profile default --format process
```

Then, once per terminal:

```
export AWS_PROFILE=cgep
```

That is the whole setup. `credential_process` is part of the AWS SDK contract, not a Terraform feature, so every tool in this course honors it.

One consequence to learn now rather than mid-lab: **re-login with `aws login --profile default`.** With `AWS_PROFILE=cgep` exported, a bare `aws login` targets `cgep` and the CLI stops you:

```
An error occurred (Configuration): Profile 'cgep' is already configured
with Credential Process credentials.
```

That is correct behavior, not a broken profile. `aws login` writes credentials into a profile, and it will not overwrite one that resolves them through a process. The session lives in `default`, so that is what you refresh. `cgep` reads through to it and needs no maintenance.

Why a profile and not an environment variable. The obvious-looking alternative is `eval "$(aws configure export-credentials --format env)"`, which copies the current credentials into the shell. It works for about fifteen minutes and then becomes the problem. Those variables are a *snapshot*: nothing updates them when the session lapses, and environment variables outrank profiles in the credential chain, so a shell holding an expired snapshot cannot be repaired by running `aws login` again. You get `ExpiredToken` from a terminal where `aws login` just reported success, which is a genuinely confusing place to be.

`credential_process` has no snapshot. Terraform re-runs the command whenever it needs credentials, so refreshing the session with `aws login` is picked up by the next Terraform command automatically. Nothing to re-export, nothing to remember.

The rule generalizes: any credential source that is not a static key file needs this bridge, which is every source you should be using.

If you already ran the `eval` line in a terminal, that shell still has the snapshot and will keep failing. Clear it before setting the profile:

```
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION
export AWS_PROFILE=cgep
```

If you must use an access key

Only when Identity Center is unavailable and `aws login` is not an option.

```
aws configure --profile cgep-lab   # paste key and secret, region us-east-1
chmod 600 ~/.aws/credentials
```

Understand what you just made: a long-lived secret, in plaintext, valid until you delete it. Rotate or remove it when the course ends:

```
aws iam list-access-keys --user-name cgep-lab
aws iam delete-access-key --user-name cgep-lab --access-key-id AKIA...
```

Never commit it. Install a guard before you can:

```
pip install detect-secrets && detect-secrets scan > .secrets.baseline
```

Step 5 The budget alarm (do not skip)

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

cat > /tmp/budget.json <<EOF
{
  "BudgetName": "cgep-lab-monthly",
  "BudgetLimit": { "Amount": "25", "Unit": "USD" },
  "TimeUnit": "MONTHLY",
  "BudgetType": "COST"
}
EOF

cat > /tmp/notifications.json <<'EOF'
[
  {
    "Notification": {
      "NotificationType": "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 50,
      "ThresholdType": "PERCENTAGE"
```

```
    },
    "Subscribers": [
      { "SubscriptionType": "EMAIL", "Address": "you@example.com" }
    ]
  },
  {
    "Notification": {
      "NotificationType": "FORECASTED",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 100,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [
      { "SubscriptionType": "EMAIL", "Address": "you@example.com" }
    ]
  }
]
EOF

aws budgets create-budget \
  --account-id "$ACCOUNT_ID" \
  --budget file:///tmp/budget.json \
  --notifications-with-subscribers file:///tmp/notifications.json
```

Two alerts: one when you have actually spent half of $25, one when the forecast says you will exceed it. The forecast alert is the useful one, because it fires while you can still act.

Budgets are a billing API and may need to run against your management account in an Organization. If you get `AccessDeniedException`, set it there instead, scoped to the sandbox account.

Step 6 Pick a region and stay in it

Every lab uses `us-east-1`. Use the same one everywhere: the state backend in Lab 2.2 and every workspace that reads it must agree, and a region mismatch produces `NoSuchBucket` errors that look like permissions errors.

If you use a different region, change it in Lab 2.2 first and carry it through consistently.

Part 2: The GCP project

Labs 2.4, 3.3, and 5.4 are GCP. You can complete the AWS track and the capstone without them, but the cross-cloud lesson is most of the point of the curriculum.

Step 7 Create the project and attach billing

If you have no Google Cloud account at all

Go to **https://cloud.google.com** and click **Get started for free**. You sign in with a Google account, so if you already have Gmail you have the identity half already. You still need:

- **A payment card**, for the same reason as AWS. Google places a small temporary authorization and releases it.
- **A phone** for verification.

New accounts get a trial credit, $300 at the time of writing, valid for ninety days. That covers this entire curriculum many times over. Google will not charge you when the trial ends unless you explicitly upgrade to a paid account, so the worst case is that things stop working rather than that you get a bill.

Two things that confuse people, and they are worth getting straight now:

A **billing account** is where the payment method lives. A **project** is where resources live. They are separate objects, and one billing account can pay for many projects. Creating a project does not attach billing automatically, which is why Step 7 links them explicitly below.

There is also an **organization**, which you get only with Google Workspace or Cloud Identity. A personal account has none, and that is fine: every GCP lab here works at project scope. Lab 5.4 says so explicitly.

When signup finishes you will have a billing account, usually named *My Billing Account*. Find its ID under **Billing** in the console, or with `gcloud billing accounts list` once the CLI is installed. It looks like `01DBB6-ECB143-6DDE9B`.

Creating the project

```
gcloud auth login

PROJECT_ID="cgep-lab-$(date +%s | tail -c 6)"   # must be globally unique
gcloud projects create "$PROJECT_ID" --name="CGE-P Lab"

gcloud billing accounts list
gcloud billing projects link "$PROJECT_ID" --billing-account=XXXXXX-XXXXXX-XXXXXX
gcloud config set project "$PROJECT_ID"
```

Billing must be attached or API calls fail with errors that do not mention billing.

Step 8 Enable the APIs

```
gcloud services enable \
  cloudkms.googleapis.com \
  iam.googleapis.com \
  iamcredentials.googleapis.com \
```

```
  cloudresourcemanager.googleapis.com \
  orgpolicy.googleapis.com \
  storage.googleapis.com \
  logging.googleapis.com \
  sts.googleapis.com
```

`orgpolicy` and `sts` are needed for Lab 5.4 and are easy to forget. Enabling takes a minute or two to propagate.

Step 9 The two authentications, which are not the same thing

This is the single most common GCP confusion, and it costs people an hour each time.

```
gcloud auth login                       # authenticates the gcloud CLI as you
gcloud auth application-default login   # writes Application Default Credentials
```

You need both. `gcloud auth login` authenticates the `gcloud` command. Terraform's google provider does not use that; it uses Application Default Credentials, a separate credential file written by the second command. Run only the first and every `gcloud` command works while every `terraform apply` fails with a confusing authentication error.

Verify both:

```
gcloud auth list                                  # your account, active
gcloud auth application-default print-access-token | head -c 20; echo "..."
```

ADC tokens expire and **Terraform will not refresh them**. When you see `reauth related error (invalid_rapt)`, run `gcloud auth application-default login` again. This is normal and will happen several times during the course.

Authenticating from a container. Both commands try to open a browser, and a container does not have one. Add `--no-launch-browser` to each:

```
gcloud auth login --no-launch-browser
gcloud auth application-default login --no-launch-browser
```

Each prints a URL to open on your own machine and waits for the code it gives you back. This is the GCP counterpart of `aws login --remote`, and you need it for both commands, not just the first.

Step 10 Roles

If you created the project you are already Owner, which is sufficient. If someone else grants you access, you need:

```
for ROLE in roles/storage.admin roles/cloudkms.admin \
            roles/orgpolicy.policyAdmin roles/iam.workloadIdentityPoolAdmin \
            roles/logging.admin roles/iam.serviceAccountAdmin; do
  gcloud projects add-iam-policy-binding "$PROJECT_ID" \
    --member="user:you@example.com" --role="$ROLE"
done
```

`roles/orgpolicy.policyAdmin` and `roles/iam.workloadIdentityPoolAdmin` are **not** included in Owner in all configurations, and Lab 5.4 fails without them with a `PERMISSION_DENIED` that does not say which role is missing.

Step 11 A GCP budget alert

```
gcloud billing budgets create \
  --billing-account=XXXXXX-XXXXXX-XXXXXX \
  --display-name="cgep-lab-monthly" \
  --budget-amount=25USD \
  --threshold-rule=percent=0.5 \
  --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0,basis=forecasted-spend
```

Needs `roles/billing.admin` on the billing account. If you do not have it, set the budget in the console; do not skip it. Lab 5.4's Data Access logs are the line item that can surprise you, at $0.50/GB ingested.

Part 3: GitHub, and your own copy of this

Everything you build gets committed, and the capstone is graded on a **public GitHub repository**, submitted as a URL plus the commit SHA you want scored. So the work has to live under your account from the first lab, not in a clone of someone else's repository that you cannot push to.

Step 12 Your account and your copy

If you do not have a GitHub account, sign up at **https://github.com**. Free is enough for everything here, including Codespaces and Actions on a public repo.

Then make your own copy of this repository. Do not just clone it: a clone points at someone else's remote and your first `git push` will be refused.

Use the template. On the repository page choose **Use this template > Create a new repository**. Name it whatever you want your portfolio to be called, and make it **public** if you intend to submit it for the capstone. This gives you a clean repository with no fork relationship, which is what you want for work you are presenting as your own.

If the template button is not offered, **Fork** does the same job; it just shows as forked from the original.

Now clone **your** copy, not this one:

```
git clone https://github.com/YOUR_USER/YOUR_REPO.git
cd YOUR_REPO
```

Check where a push would go. The URL should have your username in it:

```
git remote -v
```

Step 12b Tell git who you are

Commits carry a name and an email. If you have never set them, git either refuses to commit or attributes your work to something unhelpful.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same address your GitHub account knows about, or GitHub will not link the commits to you and your contribution graph will stay empty.

Inside a container these settings do not carry over from your machine, so set them there too the first time.

Step 12c Authenticate so you can push

The easiest route, and `gh` is already in the container:

```
gh auth login
```

Choose **GitHub.com**, then **HTTPS**, and let it authenticate git for you. From a container with no browser, pick the device-code option it offers rather than the browser one.

Verify:

```
gh auth status
```

Password authentication over HTTPS has not worked for years. If you skip this and try to push, you get an authentication failure that does not explain that a token, not a password, is what is wanted.

Step 12d What you are building

The labs are not eleven separate exercises. They are one repository, built up a piece at a time, and the capstone is what it looks like finished. Nothing is thrown away between labs, and nothing later goes hunting in a previous lab's folder for something it needs.

```
your-repo/
├── terraform/
│   ├── bootstrap/                 Lab 2.2   the state backend, keeps local state
│   ├── primitives/
│   │   ├── compliant-s3/          Lab 2.3   the pattern every bucket reuses
│   │   ├── compliant-gcs-bucket/  Lab 2.4   the same idea on GCP
│   │   └── evidence-vault/        Lab 2.5   Object Lock storage
│   └── baselines/
│       ├── aws/                   Lab 5.2   CloudTrail, Config, Security Hub
│       └── gcp/                   Lab 5.4   Org Policy, federation, audit logs
├── policies/                      Lab 3.3, extended by Lab 3.4
│   ├── *.rego
│   └── tests/
├── scripts/
│   ├── capture-evidence.sh        Lab 2.5
│   ├── mutation-test.sh           Lab 3.3
│   ├── policy-gate.sh             Lab 3.4
│   ├── verify-evidence.sh         Lab 4.4
│   ├── gen-oscal-requirements.py  Lab 6.1
│   └── verify-oscal-graph.sh      Lab 6.1
├── queries/                       Lab 5.2   the Athena SQL that earns AU-6
├── oscal/
│   ├── components/                Lab 6.1
│   └── profiles/                  Lab 6.1
├── .github/workflows/grc-gate.yml  Lab 4.3, extended by Lab 4.4
├── evidence/
│   └── lab-2-2/ ... lab-6-1/      every lab deposits its proof here
└── cgep.env                       your values, gitignored
```

Read it as a dependency graph rather than a filing system. Lab 2.2's backend is where every later workspace keeps its state. Lab 2.3's primitive is the pattern Lab 2.5's vault and the capstone's buckets both reuse. Lab 3.3's policies are what Lab 3.4 gates on and Lab 4.3 runs in CI. Lab 6.1's OSCAL component points at evidence produced by all of them. **Each lab consumes what the last one produced**, which is why the order in the reading list is not a suggestion.

Your evidence goes to your repository, not to anyone else's. Because you took a copy in Part 3, the `evidence/` you build belongs to you: `git push` goes to your remote, and nothing you capture can reach the repository you copied from. That is the whole reason for taking a copy rather than cloning someone else's.

If you are reading this in the source repository rather than your own copy, note that it deliberately ships **no** evidence. A template hands the next person a snapshot, and one built from someone else's account IDs and bucket names would be worse than an empty one.

A note on this repository's own layout. These notes keep a worked copy of every lab under `reference/lab-2-2/`, `reference/lab-2-3/` and so on, one directory per lab so each can be run and tested independently. That is a reference implementation, not the shape you are building. When a guide says `terraform/primitives/compliant-s3`, that is your repository; when a command says `../lab-2-3`, that is the reference copy. Both work, because Terraform cares about finding a workspace rather than where you keep it. Pick one and stay in it.

Part 4: The toolchain

Ten tools, each with its own installer. There are two ways to get them, and the first one is strongly preferred unless you specifically want the practice.

Step 13a The container (recommended)

The repository ships a devcontainer with the whole toolchain pinned to the versions this course was written against. It builds natively on both `amd64` and `arm64`, which matters: every Mac sold since 2020 is `arm64`, and the manual instructions below are `amd64` only.

There are two ways to run it from the same definition. Pick one.

Option 1: in your browser, nothing installed

On the repository page on GitHub, click **Code**, choose the **Codespaces** tab, then **Create codespace on main**. That is the whole thing. GitHub builds the container on its own machines and gives you VS Code in a browser tab.

This is the answer if you are on Windows, on a work laptop you cannot install software on, or you simply do not want ten new binaries on your machine. The free tier covers far more than this course needs.

Option 2: on your own machine

You need three pieces first:

1. **Docker.** Install Docker Desktop and start it. It must be *running*, not merely installed; the whale icon should be in your menu bar or system tray.
2. **VS Code.**
3. **The Dev Containers extension.** In VS Code open the Extensions panel (`Ctrl+Shift+X`, or `Cmd+Shift+X` on a Mac), search for **Dev Containers**, and install the one published by Microsoft. Its identifier is `ms-vscode-remote.remote-containers`.

You should already have your own copy from Part 3. If you skipped it, go back: cloning this repository directly leaves you unable to push, and your portfolio needs somewhere to live.

Now the step that catches almost everyone:

> **Open the repository folder itself, on its own. File > Open Folder**, then select the folder that contains `README.md` and `.devcontainer`. Not the folder above it, and not a saved multi-root workspace containing several projects.

VS Code only offers to reopen in a container when `.devcontainer` sits at the top level of what you have open. Open your whole projects directory, or a workspace holding six repositories, and it will not find the configuration and will not offer anything.

With the folder open, a prompt appears in the bottom right: **"Folder contains a Dev Container configuration file. Reopen folder to develop in a container."** Click **Reopen in Container**.

If the prompt does not appear, press `Ctrl+Shift+P` (`Cmd+Shift+P` on a Mac), type **Dev Containers: Reopen in Container**, and press Enter.

> **If VS Code offers "Add Dev Container Configuration Files", say no.** That means it did not find the existing configuration, and it is offering to create a brand new empty one. Accepting gives you a second, generic container that does not have any of the tools in it. Close the dialog and check you opened the repository folder on its own, per the box above.

The first start takes a few minutes. It is building the image and pulling roughly 3 GB of tooling. A long quiet pause is the build working, not a hang; later starts reuse the image and open in seconds. You can watch it by clicking **Starting Dev Container (show log)** in the notification.

The container has its own `~/.aws`, separate from your machine's, kept on a named volume so it survives rebuilds. It starts empty, so the container writes the `[profile cgep]` stanza for you the first time it starts. It writes no credentials: you still run `aws login` yourself, and inside a container that means `aws login --remote --profile default`.

Log in from inside the container, not on your machine. This is the right way round, and the reasoning is worth following because the tempting alternative is the insecure one.

The alternative is to log in on your machine and bind-mount `~/.aws` into the container with something like `-v ~/.aws:/home/vscode/.aws`. It is a common pattern and you should not use it here. Your real `~/.aws` accumulates every profile you have ever configured, including work and production accounts. Bind mounting it hands all of them to whatever runs in that container, which includes Terraform providers and any tool a lab downloads. The container needs one sandbox account, so give it exactly that and nothing else.

A dedicated volume is therefore the smaller blast radius, not the larger one.

Know what is in that volume, though. `aws login` caches more than the fifteen minute session. Alongside the access token it stores a refresh token that outlives it, and a DPoP key that binds the two together. The binding means a stolen refresh token is not usable on its own, which is a real improvement over a bearer token, but the pair sitting together in one volume is still a credential. Treat it like one:

```
aws logout --profile default     # clears the cached login
```

And when you are finished with the course entirely, remove the volumes rather than leaving them on the machine:

```
docker volume rm cgep-aws cgep-gcloud
```

What is deliberately absent from all of this is a long-lived access key. There is no `aws_access_key_id` anywhere in the container, on your machine, or in the image. Re-authenticating every fifteen minutes is mildly annoying and it is the reason a leak of this volume expires on its own.

When it finishes, the green box at the bottom left of VS Code reads **Dev Container: CGE-P Labs**, and a terminal there starts you in `/workspaces/cgep-labs-study-notes`. That path is the same folder as on your own machine: edits inside the container are edits to your real files, and your git history is the same one.

Either way you land in a shell with `terraform`, `aws`, `gcloud`, `opa`, `conftest`, `trivy`, `cosign`, `trestle`, `jq` and `gh` already present and on your `PATH`. Skip to Step 14 and verify. `AWS_PROFILE` is already set to `cgep`; you still create the profile itself in Step 4 and still run `aws login`, because the container deliberately contains no credentials.

Logging in from a container. `aws login` opens a browser, and a container does not have one. Use `aws login --remote --profile default`, which prints a URL to open on your own machine and prompts for the code it shows you. Everything else behaves identically. Your `~/.aws` and gcloud config live on named volumes, so rebuilding the container does not cost you another login.

The same applies on the GCP side, and there it bites twice, because `gcloud auth login` and `gcloud auth application-default login` each open a browser separately. Add `--no-launch-browser` to both. This is covered in Step 9.

When the container does not cooperate

"Reopen in Container" is not offered anywhere. Three causes, in order of likelihood. You opened the wrong folder, so re-read the box above. The Dev Containers extension is not installed. Or you have a multi-root workspace open, which you can tell because the Explorer heading reads something like `MYSTUFF (WORKSPACE)` instead of the repository name.

It offers to add configuration files instead. Same cause, same fix. Say no.

An error mentioning Docker, the daemon, or a socket. Docker Desktop is not running, or cannot be reached. Start it and wait for the whale icon to settle. On Linux, note that a VS Code installed as a snap or flatpak is sandboxed and sometimes cannot see Docker Desktop's socket at all; if that is your situation, the `.deb`/`.rpm` build of VS Code avoids it.

You want a shell without VS Code. Running the image directly works, but a bare `docker run` gives you a container with none of your files in it and nothing that survives exit. You have to mount the repository and the credential volumes yourself:

Build it once, giving it a name you can refer to:

```
docker build -t cgep-labs .devcontainer
```

Then run it, from the repository folder:

```
docker run --rm -it \
  -v "$PWD":/workspaces/cgep-labs-study-notes \
  -v cgep-aws:/home/vscode/.aws \
  -v cgep-gcloud:/home/vscode/.config/gcloud \
  -w /workspaces/cgep-labs-study-notes \
  -e AWS_PROFILE=cgep \
  cgep-labs
```

`--rm` throws the container away when you exit, which is fine: your files are on your machine and your logins are in the two named volumes, so nothing you care about lives in the container itself.

Everything works but your cloud login is gone. Check you are using the named volumes above. Without them each container starts with an empty `~/.aws` and you will be logging in every time.

Step 13b Installing by hand

Do this if you want to know exactly what lands on your machine, which is a reasonable thing to want in a security course.

This path is Ubuntu 24.04 on `amd64` only. Every download below names `linux_amd64` explicitly. On an Apple Silicon Mac or an ARM machine each one is a different filename, and on Windows the shell itself differs; use the container, or WSL2 with Ubuntu, rather than translating fourteen URLs by hand.

`~/.local/bin` must be on your `PATH`; every install below is sudo-free.

```
mkdir -p ~/.local/bin
case ":$PATH:" in *":$HOME/.local/bin:"*) : ;; *) echo 'export PATH="$HOME/.local/bin:$PATH"' >>
~/.bashrc && export PATH="$HOME/.local/bin:$PATH" ;; esac
```

Terraform (check the current version at `https://checkpoint-api.hashicorp.com/v1/check/terraform`):

```
TF_VERSION=1.15.8
cd /tmp
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_linux_amd6
4.zip"
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_SHA256SUMS
"
sha256sum --ignore-missing -c "terraform_${TF_VERSION}_SHA256SUMS"
unzip -o "terraform_${TF_VERSION}_linux_amd64.zip" terraform -d ~/.local/bin
```

Note `unzip ... terraform -d`, naming the one file you want. Unzipping the whole archive drops a `LICENSE.txt` into your bin directory.

Verify the signature too. You are taking a course about supply chain evidence:

```
curl -fsSL https://www.hashicorp.com/.well-known/pgp-key.txt | gpg --import
curl -fsSLO "https://releases.hashicorp.com/terraform/${TF_VERSION}/terraform_${TF_VERSION}_SHA256SUMS
.sig"
gpg --verify "terraform_${TF_VERSION}_SHA256SUMS.sig" "terraform_${TF_VERSION}_SHA256SUMS"
```

`Good signature from "HashiCorp Security..."` is what you want. The "key is not certified" warning is expected and only means you have not assigned local trust.

AWS CLI v2. Do not install this from `apt`; the packaged `awscli` is v1, and `aws configure export-credentials` is v2-only.

```
cd /tmp
curl -fsSL "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip
./aws/install --install-dir ~/.local/aws-cli --bin-dir ~/.local/bin
```

Add `--update` to that last command to upgrade later.

gcloud. The widely-cited `curl https://sdk.cloud.google.com | bash` is interactive and stops to ask questions. Use the tarball with explicit flags instead:

```
cd /tmp
curl -fsSL https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-cli-linux-
x86_64.tar.gz -o gcloud.tar.gz
tar -xzf gcloud.tar.gz -C ~/
~/google-cloud-sdk/install.sh --quiet --path-update true --usage-reporting false --command-completion
true
exec -l $SHELL
```

About 92MB. `--path-update true` appends to your shell profile, so start a new shell before `gcloud` resolves.

OPA, Conftest, Trivy, Cosign, gh. These versions were current as of this writing. Check for newer with the loop underneath before you install.

```
cd /tmp
curl -fsSL https://github.com/open-policy-agent/opa/releases/download/v1.19.1/opa_linux_amd64_static -
o ~/.local/bin/opa
chmod +x ~/.local/bin/opa

curl -fsSL https://github.com/open-policy-agent/conftest/releases/download/v0.69.0/
conftest_0.69.0_Linux_x86_64.tar.gz | tar -xz -C ~/.local/bin conftest

curl -fsSL https://github.com/aquasecurity/trivy/releases/download/v0.74.0/
trivy_0.74.0_Linux-64bit.tar.gz | tar -xz -C ~/.local/bin trivy

curl -fsSL https://github.com/sigstore/cosign/releases/download/v3.1.3/cosign-linux-amd64 -o ~/.local/
bin/cosign
chmod +x ~/.local/bin/cosign

curl -fsSL https://github.com/cli/cli/releases/download/v2.98.0/gh_2.98.0_linux_amd64.tar.gz | tar -
xz -C ~/.local/bin --strip-components=2 gh_2.98.0_linux_amd64/bin/gh
```

```
for r in open-policy-agent/opa open-policy-agent/conftest aquasecurity/trivy sigstore/cosign cli/cli;
do
  printf '%-30s %s\n' "$r" "$(curl -sS "https://api.github.com/repos/$r/releases/latest" | jq -r .tag_
name)"
done
```

Each of those tar extractions names the single binary to pull out. Extracting the whole archive scatters `LICENSE` and `README` files into your bin directory.

OPA 1.x. OPA 1.0 made the `rego.v1` syntax the default. Every policy in this curriculum declares `import rego.v1` explicitly, which is valid in 1.x and required by 0.x, so the library runs on both. Inherited v0-syntax policies need `--v0-compatible` or a migration.

Cosign 3.x. Cosign went 2.x to 3.x. The flags Lab 4.4 depends on (`sign-blob --bundle`, and `verify-blob --bundle --certificate-identity-regexp --certificate-oidc-issuer`) are all present in 3.1.3, so the lab is unaffected. Confirm with `cosign verify-blob --help` if you install a later major version.

trestle. Ubuntu 24.04 and other recent distributions enforce PEP 668, so plain `pip install --user` fails with `error: externally-managed-environment`. Use `pipx`:

```
sudo apt install -y pipx jq git unzip curl   # most are present already
pipx install compliance-trestle
```

`pipx` gives trestle its own virtualenv and links the binary into `~/.local/bin`. If you would rather not use pipx: `python3 -m venv ~/.venvs/trestle && ~/.venvs/trestle/bin/pip install compliance-trestle`, then link the binary onto your `PATH`.

Check which OSCAL version your trestle targets. It decides what you write in Lab 6.1.

```
trestle version
# Trestle version v5.0.0 based on OSCAL version 1.2.1
```

Your component definition, your profile, and the NIST catalog you import must all agree on that number. Lab 6.1 uses **1.2.1** to match trestle 5.x. If your trestle reports something different, use what it reports; a mismatch is the most common `trestle validate` failure.

`gh`, the GitHub CLI, per **cli.github.com**, then `gh auth login`.

Step 14 Verify the whole toolchain

This one ships as a file, so you can run it instead of pasting it:

```
bash reference/lab-0-1/scripts/check-prereqs.sh
```

It is reproduced here so you can read what it checks before you run it.

```
#!/usr/bin/env bash
# scripts/check-prereqs.sh
FAIL=0
check() {
  if command -v "$1" >/dev/null 2>&1; then
    printf '  %-12s %s\n' "$1" "$($2 2>&1 | head -1)"
  else
    printf '  %-12s MISSING\n' "$1"; FAIL=1
  fi
}

echo "=== toolchain ==="
check terraform "terraform version"
check aws       "aws --version"
check gcloud    "gcloud version"
check opa       "opa version"
```

```
check conftest  "conftest --version"
check trivy     "trivy --version"
check cosign    "cosign version"
check trestle   "trestle version"
check jq        "jq --version"
check gh        "gh --version"

echo "=== AWS ==="
aws sts get-caller-identity --profile "${AWS_PROFILE:-cgep}" 2>&1 | head -4 || FAIL=1

echo "=== GCP ==="
gcloud config get-value project 2>&1 | head -1
gcloud auth application-default print-access-token >/dev/null 2>&1 \
  && echo "  ADC: OK" || { echo "  ADC: MISSING (run: gcloud auth application-default login)";
FAIL=1; }

echo
[[ $FAIL -eq 0 ]] && echo "PREREQS OK" || { echo "PREREQS INCOMPLETE"; exit 1; }
```

```
chmod +x scripts/check-prereqs.sh && bash scripts/check-prereqs.sh
```

`PREREQS OK` means you are ready for Lab 2.2.

Part 5: Set your values once

Across the course you supply the same handful of values over and over. `project_name` is asked for by four labs, `aws_region` by five. Typing them each time is how you end up with a bucket in the wrong region, or two labs that disagree about what your project is called.

Terraform reads any environment variable named `TF_VAR_<variable>` as an input. So you can set every one of them once, in a file you source at the start of each session, and stop passing them to individual commands entirely.

Step 15 Create `cgep.env`

In the repository root:

```
cat > cgep.env <<'EOF'
# Sourced at the start of every session. Not committed: it names your account,
# your project, and your repository.

# ---- AWS, needed from Lab 2.2 onward ----
export AWS_PROFILE=cgep
export AWS_REGION=us-east-1
export TF_VAR_aws_region="$AWS_REGION"

# 3 to 21 lowercase letters, digits and hyphens. Becomes your bucket prefix,
```

```
# so it has to be globally unique-ish. Your own name or initials work well.
export TF_VAR_project_name=cgep-lab

# dev, staging or prod. Drives the Environment tag.
export TF_VAR_environment=dev

# ---- GCP, needed for Labs 2.4, 3.3 and 5.4. Leave blank for the AWS track ----
export TF_VAR_gcp_project=
export TF_VAR_gcp_region=us-central1
export CLOUDSDK_CORE_PROJECT="$TF_VAR_gcp_project"

# ---- GitHub, needed for Labs 4.3, 4.4 and 5.4 ----
export TF_VAR_github_org=
export TF_VAR_github_repo=

# ---- Filled in by labs as you finish them. Leave these alone for now. ----
# Lab 2.2 gives you the state backend:
# export TF_VAR_state_bucket=
# export TF_VAR_state_kms_arn=
# Lab 2.5 gives you the evidence vault:
# export TF_VAR_evidence_vault_arn=
EOF
```

Fill in the blanks you know, leave the rest. Then make sure it is never committed, because it names your account and your repository:

```
grep -qxF 'cgep.env' .gitignore || echo 'cgep.env' >> .gitignore
```

Step 16 Source it, every session

```
source cgep.env
```

That is the first command of every working session, before any `terraform` or `aws` command. A convenient check that it worked:

```
echo "profile=$AWS_PROFILE project=$TF_VAR_project_name region=$AWS_REGION"
```

From here on, `terraform plan` and `terraform apply` find their inputs on their own. **You do not need a `terraform.tfvars` and you do not need `-var` flags.** Where a lab shows those, they are the alternative for anyone not using this file; if you sourced `cgep.env` you can leave them off.

In the container this is already done for you. The devcontainer's `postStartCommand` adds the line on every start, pointing at wherever your repository is actually mounted. That placement is deliberate: `~/.bashrc` lives in the image rather than in a mounted volume, so a container rebuild throws it away, and `postStartCommand` runs again afterwards and puts it back.

Outside a container, add it yourself. Derive the path rather than typing one: your repository is named whatever you named it when you created it from the template, so a hardcoded path is a line that silently does nothing.

```
REPO=$(git rev-parse --show-toplevel)
grep -q cgep.env ~/.bashrc \
  || echo "if [ -f \"$REPO/cgep.env\" ]; then . \"$REPO/cgep.env\"; fi" >> ~/.bashrc
```

The `if` form matters more than it looks. Writing `[ -f ... ] && . ...` leaves your shell's exit status at 1 whenever the file is absent, which shows up in prompts that display it and in any script that checks `$?` early.

> **When Terraform prompts you for a variable, that is this.** A fresh shell, or a rebuilt container, and nothing sourced. Answering the prompt works once and teaches you nothing. Press Ctrl+C, source the file, and run it again.

Step 17 Add values as labs produce them

Some values do not exist until a lab creates them. Lab 2.2 produces the state bucket and its key; Lab 2.5 produces the evidence vault. Each of those labs tells you to append its outputs, in this shape:

```
cd reference/lab-2-2
# Delete any previous value first, so re-running a lab updates cgep.env rather
# than stacking a second export that silently shadows the first. This runs
# before the redirect on purpose: sed -i replaces the file, and an already-open
# >> would keep writing to the old inode.
sed -i '/^export TF_VAR_state_bucket=/d;/^export TF_VAR_state_kms_arn=/d' ../../cgep.env
{
  echo "export TF_VAR_state_bucket=$(terraform output -raw state_bucket)"
  echo "export TF_VAR_state_kms_arn=$(terraform output -raw state_kms_key_arn)"
} >> ../../cgep.env
cd ../..
source cgep.env
```

Reading them out of `terraform output` rather than copying them from your terminal is deliberate. Both strings contain your account ID and a random suffix, and transcription is exactly where this goes wrong.

Part 6: Habits that will save you

Set `AWS_PROFILE` at the start of every session, which `cgep.env` does for you. The `[profile cgep]` entry points at `--profile default`, because that is where `aws login` writes. If you configured a named login profile instead, point it there.

Never export credentials into the shell. The one habit that will cost you an afternoon is `eval "$(aws configure export-credentials --format env)"`. Environment variables outrank profiles, so the moment that snapshot expires your shell is stuck: `aws login` succeeds and Terraform keeps failing, because the provider never looks past the dead variables. If a shell is already in that state, `unset` the four variables rather than trying to refresh them.

Check which identity you are actually using, especially after a re-login:

```
aws sts get-caller-identity --query Arn --output text
```

An `Arn` ending in `:root` means stop and re-authenticate as your IAM user.

Never commit credentials. The repository's `.gitignore` excludes `*.tfvars`, `*.tfstate`, `backend.hcl` and `cgep.env` for this reason. Add a scanner before you need one.

Your evidence is your portfolio, so commit it. Every lab's submission checklist asks for files under `evidence/lab-X-Y/`, and the capstone is graded on a public repository submitted by URL and commit SHA. Those files are the point. They are deliberately not ignored.

Three things to be deliberate about before you make that repository public:

- **Terraform state is the one real risk.** State records every attribute the provider stored, including ones marked sensitive, which is the trap Lab 2.5 is built around. `evidence/**/state.json` is gitignored for that reason. When a checklist asks for it, look at the file first and then add it on purpose:

  ```
  grep -iE 'password|secret|private_key|token' evidence/lab-2-3/state.json
  git add -f evidence/lab-2-3/state.json
  ```

  For Lab 2.3 that file holds no secrets, which is why the checklist asks for it. Do not assume the same of a workspace that manages a database or a key pair.

- **Your account ID will be in there**, in every ARN, and there is no avoiding it if you publish evidence at all. It is not a credential and it is not a breach, but it does help someone enumerate your roles. Decide once whether you are comfortable with that; if you are not, keep the repository private and share it with reviewers directly.

- **Bucket names are globally unique and yours.** Publishing them tells the world which buckets to probe. The controls you built are exactly what makes that survivable, which is worth saying in your write-up rather than hiding.

Destroy what you finish, except the Lab 2.2 state backend, which everything else depends on. Destroy that last, after Lab 6.1, and only after every other workspace is gone. Deleting the state bucket while resources still exist strands them: no state means Terraform can no longer destroy them, and you are deleting things by hand in the console.

Check your bill weekly:

```
aws ce get-cost-and-usage \
  --time-period Start=$(date -u -d '30 days ago' +%Y-%m-%d),End=$(date -u +%Y-%m-%d) \
  --granularity MONTHLY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE \
  --query 'ResultsByTime[].Groups[?Metrics.UnblendedCost.Amount>`0.01`].
[Keys[0],Metrics.UnblendedCost.Amount]' \
  --output table
```

Cost Explorer must be enabled once in the console before this API returns data, and it can take 24 hours to populate.

Troubleshooting

- `Credentials were refreshed, but the refreshed credentials are still expired.` Two different causes. If it appears seconds after `aws login`, it is a transient cache race: `aws login` returns your prompt as soon as it writes the profile, while the CLI's own cache still holds the previous entry. Re-run the command. If it persists, your shell is holding an expired snapshot from `export-credentials --format env`, which outranks the profile: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION` and re-run.
- `No valid credential sources found` plus `no EC2 IMDS role found` from Terraform, while `aws sts get-caller-identity` works fine. Nothing is wrong with EC2; that is just where the credential chain gave up. Neither `aws login` nor Identity Center writes credentials the provider can read. Set up the `credential_process` profile from Part 1 and `export AWS_PROFILE=cgep`. **This is the most common error in the entire course.**
- `failed to find SSO session section` from Terraform. Same cause, different wording, same fix.
- `ExpiredToken` mid-lab. Your session lapsed. Re-run `aws login` (or `aws sso login` on the Identity Center path), then re-export. Short-lived credentials expiring is the feature, not a fault.
- `aws: command not found` after installing. `~/.local/bin` is not on `PATH`, or you need a new shell.
- `aws configure export-credentials: Invalid choice.` You have AWS CLI v1. Install v2; do not use `apt`.
- `reauth related error (invalid_rapt)` from Terraform on GCP. Re-run `gcloud auth application-default login`.
- `PERMISSION_DENIED` **on Org Policy or WIF** despite being Owner. Owner does not always include `roles/orgpolicy.policyAdmin` or `roles/iam.workloadIdentityPoolAdmin`. Grant them explicitly.
- `billing account not found.` The project has no billing attached. `gcloud billing projects link`.
- **Budget creation returns** `AccessDeniedException.` Budgets are a billing API. In an Organization, create it from the management account.

Teardown, when you finish the course

In this order:

1. Destroy every lab workspace except the Lab 2.2 bootstrap.
2. Destroy the Lab 2.2 bootstrap last.
3. Check for orphans: `aws resourcegroupstaggingapi get-resources --tag-filters Key=ComplianceScope,Values=cge-p-lab`. This is what the `ComplianceScope` tag was for all along, and finding your own strays with it is a good final exercise.
4. Schedule KMS key deletion for anything left. Keys bill through their waiting period.
5. GCP: `gcloud projects delete "$PROJECT_ID"`. Deleting the project is the cleanest teardown, and it is 30-day recoverable.
6. If you created a dedicated AWS member account, close it. AWS retains it in `SUSPENDED` for 90 days.

7. Delete any IAM user access keys you created in Path B.

Leave the budget alarm in place until the account is closed. It is the last thing that will tell you about something you forgot.

Revision history

These notes

- New document. Sandbox setup, credentials, toolchain and budget alarms were previously scattered across each lab's prerequisites.
- `aws login` is the recommended credential path, ahead of IAM Identity Center, with access keys demoted to last resort.
- Creating a non-root IAM user comes before the credential step, because `aws login` will otherwise offer the root session without flagging it.
- Terraform is bridged to `aws login` with a `credential_process` profile rather than by exporting credentials into the shell. The profile resolves credentials per command, so a re-login is picked up automatically; an exported snapshot goes stale after about fifteen minutes and then outranks every attempt to fix it.

The official labs

Did not exist. Prerequisites were listed per lab.